smart waste compliance monitoring and assessment syste

Free users can only access their 5 most recently updated projects

swcmas_tar.gz

GZ

"Earlier on, we were looki ng to eradicate the need f or sensors, but as it appea rs, looking at the current t echnological advancemen t, we actually need thos…

PASTED

go through this project comprehensively . we had started the implementation but now we want to change the waste monitoring . instead of relying on before and after images, we want to use sensors to detect if waste has beeen collected or not. i want us to plan first.

Ran 6 commands, viewed a file >

Now I have a thorough picture of the project. Let me build the planning document.

Read scheduler and socket utility >

Now I have everything I need. Let me build a comprehensive planning document.
Now I have enough to build a comprehensive plan. Let me create the planning document.

Sensor mig
Replacing before/after ph

⚠ Free users can only access their 5 most recently updated projects

What's changing

Old: Image-based
beforePhoto + afterPhoto + mock AI score

→

New: Sensor-based
ultrasonic fill level + weight delta + timestamp

MVP hardware per bin (≈ $20)

ESP32	HC-SR04	Wi-Fi	Power
Microcontroller ($5–10)	Ultrasonic sensor ($3–5)	Built-in on ESP32	USB or solar

Data flow — how sensor readings reach the backend

Bin (ESP32)
Reads distance every 5 min

→

MQTT / HTTP
JSON payload over Wi-Fi

→

Node.js backend
POST /api/v1/sensors/reading

→

MongoDB
SensorReading model

How we detect waste collection from sensor data

Collection detection algorithm (sensor.service.js)

1. Fill level before collection scheduled time → e.g. 85% full
2. Fill level after expected collection window → e.g. 12% full
3. If delta ≥ threshold (e.g. 60%) AND drop happens within ±2hr of scheduled time → COLLECTED
4. If no significant drop after 6hr past schedule → MISSED (trigger anomaly alert)

Backend changes

New: SensorReading model
apartment: ObjectId
fillLevelPercent: Number (0–100)
rawDistanceCm: Number
binHeightCm: Number
recordedAt: Date

Changed: Collection model
REMOVE beforePhoto, afterPhoto
ADD preSensorReadingId: ObjectId
ADD postSensorReadingId: ObjectId
ADD fillDeltaPercent: Number
ADD sensorVerified: Boolean

New: sensor.routes.js
POST /sensors/reading (ESP32 posts here)
GET /sensors/:apartmentId (dashboard)
GET /sensors/:apartmentId/history

Changed: collection.service.js
REMOVE AI mock photo comparison
ADD sensor-based auto-verify on complete
ADD fillDelta → verificationScore.sensor

New: sensor.service.js
saveSensorReading(apartmentId, payload)
getLatestReading(apartmentId)
detectCollectionFromSensor(collectionId)

Changed: anomaly.service.js
ADD sensor_tamper (sudden spike up)
ADD no_sensor_data (bin offline alert)
KEEP collection_gap, suspicious_duration

Verification score — replacing the image component

Updated scoring breakdown (total still 100 pts)
GPS proximity: 20 pts | Time spent: 20 pts | Landlord confirm: 20 pts | Resident feedback: 20 pts
Sensor delta (replaces image): 20 pts → fillDeltaPercent ≥ 60% = full score, prorated below

Frontend changes

CollectorDashboard
Remove photo upload UI
Show live fill level gauge
Show sensor status badge

VerificationPage
Replace before/after img
Show pre/post fill level
Show delta chart

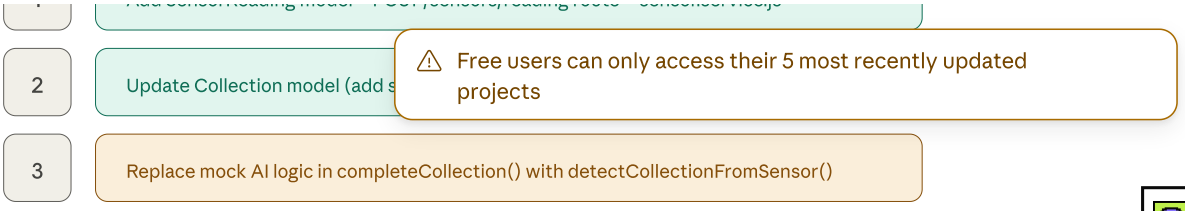
AdminDashboard
Sensor health overview
Offline bin alerts
Fill level heatmap

New sensor API — what ESP32 posts

POST /api/v1/sensors/reading
Body: { "apartmentId": "...", "rawDistanceCm": 14.2, "binHeightCm": 80 }
Server computes: fillLevelPercent = ((binHeightCm - rawDistanceCm) / binHeightCm) × 100
Auth: static device API key in header X-Device-Key: <key>

Implementation order

1 Add SensorReading model + POST /sensors/reading route + sensor.service.js



Here's the full picture after going through every file.

Current state — what you built

The system currently has collectors upload a `beforePhoto` and `afterPhoto` when they start and complete a collection. The "verification" is a mock: `Math.random() * 100` determines whether an AI "auto-verified" it. The real scoring in `verifyCollection()` gives 20 points to the image component simply if both photos exist — there's no actual analysis. The `FieldOpsWizard` on the frontend drives this photo upload flow. This is what we're replacing.

The new detection approach

An ESP32 with an ultrasonic sensor (HC-SR04) mounts inside each bin lid and measures the distance to the waste surface every few minutes. The system then infers collection by detecting a significant drop in fill level (e.g. from 85% full to 12% full) within a time window around the scheduled collection time. No human input, no photos, no fake AI.

What needs to change — layer by layer

New things to build:

- `SensorReading` model — stores `apartmentId`, `rawDistanceCm`, `binHeightCm`, `fillLevelPercent`, `recordedAt`
- `sensor.service.js` — three core functions: `saveSensorReading()`, `getLatestReading()`, and `detectCollectionFromSensor()` (the algorithm that compares pre- and post-readings around the scheduled time)
- `sensor.routes.js` + `sensor.controller.js` — `POST /api/v1/sensors/reading` (ESP32 calls this), `GET /api/v1/sensors/:apartmentId` for dashboard
- A static device API key system (simple header `X-Device-Key`) to authenticate the ESP32 without giving it a JWT

Things to change:

- `Collection` model — add `preSensorReadingId`, `postSensorReadingId`, `fillDeltaPercent`, `sensorVerified`. Mark `beforePhoto` / `afterPhoto` as optional/deprecated (don't delete yet — preserve old data)
- `collection.service.js → completeCollection()` — remove the `Math.random()` mock, replace with a call to `detectCollectionFromSensor()`. The verificationScore's `image: 20` component becomes `sensor: 20`, prorated by how large the fill delta was
- `anomaly.service.js → runDetections()` — add two new anomaly types: `sensor_tamper` (fill level suddenly spikes up, which shouldn't happen without physical interference) and `no_sensor_data` (a bin hasn't reported in over 24 hours)
- `config/index.js` — add `sensor_tamper` and `no_sensor_data` to `anomalyTypes`

Frontend changes:

- `CollectorDashboard` / `FieldOpsWiza` level gauge showing the current sensor reading for the bin being serviced
- `VerificationPage` — instead of before/after images, show a mini chart of fill level over time with the pre-collection reading, the drop, and the post-collection reading highlighted
- `AdminDashboard` / `MapViewPage` — add sensor health indicators (bins online/offline) and fill level overlays on the map

⚠ Free users can only access their 5 most recently updated projects

---

The detection algorithm (most important part)

When `completeCollection()` is called:

1. Look up the latest `SensorReading` for this bin that was recorded **before** `collection.startTime`
2. Look up the latest reading recorded **after** `collection.endTime`
3. Compute `fillDelta = preFill - postFill`
4. If `fillDelta >= 60%` (configurable threshold) → `sensorVerified = true`, sensor score = 20
5. If delta is 30–60% → prorated score, flag for landlord review
6. If delta < 30% or no post-reading available → score = 0, trigger anomaly

The implementation order from the diagram is the right sequence — get the sensor data ingestion working first (steps 1-2), then wire it into the collection flow (step 3), then update the frontend.

Want to start with the `SensorReading` model and the `POST /sensors/reading` endpoint?

give me a full detailed prompt for implemting this entire system

Here's a comprehensive implementation prompt you can use:

SWCMAS Sensor Migration — Full Implementation Prompt

You are implementing a sensor-based waste collection verification system for SWCMAS (Smart Waste Collection Management and Analytics System). The project is a Node.js/Express backend with MongoDB (Mongoose) and a React/Vite frontend using Tailwind CSS. The codebase already exists — you are making targeted additions and modifications.

The goal: Replace the current image-based collection verification (before/after photos + mock AI scoring) with ultrasonic sensor data from ESP32 microcontrollers mounted in waste bins. Waste collection is detected by measuring a significant drop in bin fill level around the scheduled collection time.

---

1. Backend — New files to create

⚠ Free users can only access their 5 most recently updated projects

`backend/src/models/SensorReading.js`

Create a Mongoose model with these fields:

```
js

apartment: { type: ObjectId, ref: 'Apartment', required: true, index: true }
rawDistanceCm: { type: Number, required: true }       // distance from sensor to waste
binHeightCm: { type: Number, required: true }         // total interior height of bin
fillLevelPercent: { type: Number, required: true, min: 0, max: 100 }  // computed: ((b
recordedAt: { type: Date, default: Date.now, index: true }
deviceId: { type: String }                            // optional ESP32 chip ID for au
batteryVoltage: { type: Number }                      // optional, for device health m
```

Add compound index on `{ apartment: 1, recordedAt: -1 }`.

`backend/src/services/sensor.service.js`

Implement five functions:

`saveSensorReading(apartmentId, payload)`

- Validate that the apartment exists (throw 404 if not)
- Compute `fillLevelPercent = ((payload.binHeightCm - payload.rawDistanceCm) / payload.binHeightCm) * 100`, clamped to 0–100
- Save and return the new `SensorReading` document

`getLatestReading(apartmentId)`

- Return the most recent `SensorReading` for the given apartment sorted by `recordedAt: -1`
- Return `null` if none exists

`getReadingHistory(apartmentId, { from, to, limit = 100 })`

- Return readings within the date range, sorted by `recordedAt: -1`, limited to `limit`

`detectCollectionFromSensor(collectionId)`

- Fetch the collection and its linked apartment
- Find the latest sensor reading recorded **before** `collection.startTime` — this is `preReading`
- Find the latest sensor reading recorded **after** `collection.endTime` but within 3 hours — this is `postReading`
- If either reading is missing, return `{ detected: false, reason: 'insufficient_sensor_data', sensorScore: 0 }`
- Compute `fillDelta = preReading.fillLevelPercent - postReading.fillLevelPercent`
- Scoring logic:
  - `fillDelta >= 60` → `sensorScore = 20`, `detected = true`

- `fillDelta >= 30` → `sensorScore = Math.round((fillDelta / 60) * 20)`, `detected = true` (partial, flag f
- `fillDelta < 30` → `sensorScor`

⚠ Free users can only access their 5 most recently updated projects

- Return `{ detected, fillDelta, preFill: preReading.fillLevelPercent, postFill: postReading.fillLevelPercent, sensorScore, preSensorReadingId: preReading._id, postSensorReadingId: postReading._id }`

`checkSensorHealth()`

- Find all active apartments (from `Apartment.find({ isActive: true })`)
- For each, check if a sensor reading exists within the last 24 hours
- Return an array of `{ apartmentId, apartmentName, status: 'online' | 'offline', lastSeenAt }`

`backend/src/controllers/sensor.controller.js`

`ingestReading` — handles `POST /sensors/reading`

- Validate `X-Device-Key` header against `process.env.SENSOR_DEVICE_KEY` (return 401 if missing/wrong)
- Call `sensor.service.saveSensorReading()`
- Emit a Socket.IO event `sensor_reading` to `county_admin` and `county_officer` rooms with the new reading payload (use `getIO()` from `utils/socket.js`)
- Return 201 with the saved reading

`getLatestReading` — handles `GET /sensors/:apartmentId`

- Auth middleware (existing JWT auth) — accessible to all roles
- Call `sensor.service.getLatestReading()`
- Return 200 with the reading or `{ fillLevelPercent: null, status: 'no_data' }` if none

`getReadingHistory` — handles `GET /sensors/:apartmentId/history`

- Accept query params `from`, `to`, `limit`
- Call `sensor.service.getReadingHistory()`
- Return paginated history

`getSensorHealth` — handles `GET /sensors/health`

- Admin/officer only (use existing role middleware)
- Call `sensor.service.checkSensorHealth()`
- Return the health array

`backend/src/routes/sensor.routes.js`

```
POST   /sensors/reading          → ingestReading   (no JWT, uses X-Device-Key
instead)
GET    /sensors/health           → getSensorHealth  (admin, officer)
```

```
GET    /sensors/:apartmentId          → getLatestReading (all authenticated roles)
GET    /sensors/:apartmentId/history
```

⚠ Free users can only access their 5 most recently updated projects

Register this router in `backend/src/app.js` under the existing API prefix.

---

2. Backend — Files to modify

`backend/src/models/Collection.js`

Add these new fields to the existing schema:

```js
preSensorReadingId: { type: ObjectId, ref: 'SensorReading' }
postSensorReadingId: { type: ObjectId, ref: 'SensorReading' }
fillDeltaPercent: { type: Number }
sensorVerified: { type: Boolean, default: false }
```

Mark `beforePhoto` and `afterPhoto` as optional (they already are — just leave them; do NOT remove them, as existing records reference them).

`backend/src/models/AnomalyAlert.js` (via **`config/index.js`**)

In `backend/src/config/index.js`, add to the `anomalyTypes` array:

- `'sensor_tamper'`
- `'no_sensor_data'`

`backend/src/services/collection.service.js`

In `completeCollection()`:

Remove the entire mock AI block:

```js
// REMOVE this entire block:
const aiVisionConfidence = Math.random() * 100;
let autoVerified = false;
if (collection.beforePhoto && collection.afterPhoto && aiVisionConfidence > 85) { ...
```

Replace it with:

```js
// After saving collection with status 'completed':
const sensorService = require('./sensor.service');
const sensorResult = await sensorService.detectCollectionFromSensor(collection._id);

if (sensorResult.detected) {
  collection.preSensorReadingId = sensorResult.preSensorReadingId;
  collection.postSensorReadingId = sensorResult.postSensorReadingId;
  collection.fillDeltaPercent = sensorResult.fillDelta;
  collection.sensorVerified = true;
```

```
  await collection.save();

  // Auto-verify if sensor score is f
  if (sensorResult.sensorScore === 20) {
    await module.exports.verifyCollection(collection._id, { verificationStatus: 'verif
    logger.info(`Sensor auto-verified collection ${collection._id} — fill delta ${sens
  } else {
    // Partial detection — notify landlord for manual review
    // (keep existing landlord notification logic here)
  }
} else {
  logger.warn(`Sensor data insufficient for collection ${collection._id}: ${sensorResu
  // Fall through to landlord notification (keep existing logic)
}
```

⚠ Free users can only access their 5 most recently updated projects

In `verifyCollection()`:

Replace the image score block:

```js
// REMOVE:
let imageScore = 0;
if (collection.beforePhoto && collection.afterPhoto) { imageScore = 20; }
else if (collection.beforePhoto || collection.afterPhoto) { imageScore = 10; }

// REPLACE WITH:
const sensorScore = collection.sensorVerified
  ? (sensorResult?.sensorScore ?? 20)   // use detected score if available
  : 0;
```

Update `verificationScore` to use `sensor` instead of `image`:

```js
collection.verificationScore = {
  gps: Math.round(gpsScore),
  timeSpent: Math.round(timeScore),
  landlord: landlordScore,
  resident: residentScore,
  sensor: sensorScore,           // replaces 'image'
  total: Math.round(gpsScore + timeScore + landlordScore + residentScore + sensorScore
};
```

> Note: The `verificationScoreSchema` in `Collection.js` currently has `image: { type: Number, default: 0, max: 20 }`. Rename this field to `sensor` in the schema. Keep `image` as an alias or legacy field on existing documents — do not run a migration unless explicitly asked.

`backend/src/services/anomaly.service.js`

In `runDetections()`, add two new detection rules after the existing ones:

Rule 6 — Sensor tamper detection:

⚠ Free users can only access their 5 most recently updated projects

```js
// A fill level increase of > 30% bet
// (waste can't appear in a bin by itself) — flag as possible sensor tamper or vandali
const readings = await SensorReading.aggregate([
  { $sort: { apartment: 1, recordedAt: 1 } },
  {
    $group: {
      _id: '$apartment',
      readings: { $push: { fill: '$fillLevelPercent', at: '$recordedAt' } }
    }
  }
]);
for (const aptReadings of readings) {
  const r = aptReadings.readings;
  for (let i = 1; i < r.length; i++) {
    const delta = r[i].fill - r[i - 1].fill;
    if (delta > 40) {  // fill went UP by more than 40% between two readings
      await createAlertIfNotExists({
        type: 'sensor_tamper',
        severity: 'high',
        entityType: 'Apartment',
        entityId: aptReadings._id,
        description: `Sensor fill level jumped +${delta.toFixed(1)}% — possible tamper
        details: { delta, at: r[i].at },
      });
    }
  }
}
```

Rule 7 — No sensor data:

```js
const healthResults = await sensorService.checkSensorHealth();
for (const h of healthResults) {
  if (h.status === 'offline') {
    await createAlertIfNotExists({
      type: 'no_sensor_data',
      severity: 'medium',
      entityType: 'Apartment',
      entityId: h.apartmentId,
      description: `No sensor readings received for over 24 hours`,
      details: { lastSeenAt: h.lastSeenAt },
    });
  }
}
```

Also add `const sensorService = require('./sensor.service');` and `const SensorReading = require('../models/SensorReading');` at the top of the file.

`backend/src/app.js`

Register the new sensor router:

```js
const sensorRoutes = require('./route
app.use(`${config.apiPrefix}/sensors`, sensorRoutes);
```

⚠ Free users can only access their 5 most recently updated projects

---

`.env` / `.env.example`

Add:

```
SENSOR_DEVICE_KEY=your-static-device-secret-here
SENSOR_FILL_THRESHOLD=60
SENSOR_PARTIAL_THRESHOLD=30
```

---

3. Frontend — Files to modify

`frontend/src/api/` — add `sensors.api.js`

```js
import axiosInstance from './axios';

export const getLatestSensorReading = (apartmentId) =>
  axiosInstance.get(`/sensors/${apartmentId}`);

export const getSensorReadingHistory = (apartmentId, params) =>
  axiosInstance.get(`/sensors/${apartmentId}/history`, { params });

export const getSensorHealth = () =>
  axiosInstance.get('/sensors/health');
```

`frontend/src/pages/dashboard/CollectorDashboard.jsx`

- Remove all UI related to photo upload (before/after photo inputs, preview images, upload state)
- Add a `FillLevelGauge` section that shows the current bin fill level for the apartment the collector is assigned to. Fetch it with `getLatestSensorReading(apartmentId)` on mount
- Display: a vertical progress bar or arc gauge showing `fillLevelPercent`, a label showing the last reading time, and a badge — green if `< 30%`, amber if `30–70%`, red if `> 70%`
- If `fillLevelPercent` is null or no data, show "No sensor data — contact admin"

`frontend/src/pages/verification/VerificationPage.jsx`

- Remove `beforePhoto` / `afterPhoto` image display
- Add a fill level comparison section:
  - Show two stat cards side by side: **Before collection** (pre fill %) and **After collection** (post fill %)
  - Show a delta value: e.g. "↓ 74% drop detected"
  - Color code: green delta = verified, amber = partial, red/grey = not detected

- If `fillDeltaPercent` is null, show "Awaiting sensor data"

Free users can only access their 5 most recently updated projects

- The existing landlord confirmation bu
  just replace `image` score label with `Se`

`frontend/src/pages/dashboard/AdminDashboard.jsx`

- Add a "Sensor health" summary card (fetch `getSensorHealth()`) showing: total bins monitored, bins online (green), bins offline (red)
- Add it alongside existing stat cards at the top of the dashboard
- Clicking "offline bins" should navigate to the AnomaliesPage filtered to `type=no_sensor_data`

`frontend/src/context/SocketContext.jsx`

Add a listener for the new `sensor_reading` event:

```js
socket.on('sensor_reading', (data) => {
  // Dispatch to a global state or context so the live map and dashboard can update
  // For now: emit a custom DOM event or use an existing state pattern in the file
});
```

`frontend/src/pages/maps/MapViewPage.jsx`

- For each apartment marker on the map, show the fill level as a colour overlay on the marker: green (< 30%), amber (30–70%), red (> 70%), grey (no data)
- Pull latest readings from the sensor health endpoint or per-apartment on map load

---

4. Environment and connectivity notes

How the ESP32 will post data:

```
POST https://your-backend/api/v1/sensors/reading
Headers:
  Content-Type: application/json
  X-Device-Key: <SENSOR_DEVICE_KEY from .env>
Body:
  {
    "apartmentId": "64abc...",
    "rawDistanceCm": 14.2,
    "binHeightCm": 80,
    "deviceId": "ESP32-A4B2",
    "batteryVoltage": 3.7
  }
```

The backend computes `fillLevelPercent` server-side so the ESP32 doesn't need to know the bin height at flash time — just post the raw distance. The `binHeightCm` can be configured per apartment in the admin panel later, but for now it's sent by the device.

⚠ Free users can only access their 5 most recently updated projects

5. What NOT to change

- Do not remove the `beforePhoto` / `afterPhoto` fields from the `Collection` model — old records depend on them
- Do not change the existing GPS scoring, time-spent scoring, landlord scoring, or resident feedback scoring logic
- Do not change the scheduler (`collectionScheduler.service.js`) — collection creation is unaffected
- Do not change the JWT auth middleware — only the sensor ingest endpoint uses the device key bypass
- Do not change invoice generation logic — it still triggers on verified collections as before
- The existing `verificationStatuses` and `collectionStatuses` enums stay the same

---

6. Testing checklist

After implementation, verify:

1. `POST /api/v1/sensors/reading` with valid `X-Device-Key` saves a reading and emits a socket event
2. `POST /api/v1/sensors/reading` without the key returns 401
3. `detectCollectionFromSensor()` returns `detected: true` when a matching pre/post reading pair shows ≥ 60% drop
4. `completeCollection()` no longer calls `Math.random()` — check logs show "Sensor auto-verified" or "Sensor data insufficient"
5. `verificationScore.sensor` (not `image`) appears in the completed collection document
6. Anomaly detection adds `no_sensor_data` alerts for apartments with no recent readings
7. The collector dashboard shows a fill gauge instead of photo upload UI
8. The verification page shows pre/post fill stats instead of images

---

Feed this prompt to your implementation session along with the relevant existing files as context. Start with items 1 and 2 (the model and service), then item 3 (the routes), then the backend modifications in order, and leave the frontend for last.